Date:- 19/11/2022

class-35 - Exception Handling part-1,

compile time error → syntax errors
java syntax

Runtime → Runtime error
↓ due to logic problem
faulty Logic

Logic that the programmer wrong

Exception → not because of Logic
but because of faulty inputs

abrupt termination

Ex:- Runtime error (Logic is wrong)

1)
```
main ()
{
    disp();
}
class A
{
    disp()
    {
    disp();
    }
}
```

In this program there are no syntax errors but this disp() method get called every time in the stack runs out space and results in stack overflow error

2) array disadvantage → fixed size
requires contagious memory locations

```
main(){
int a[]=new int[55];
```

4*55 = 220 bytes should be continously available in Heap

```
int a[]=new int[9999999];
```

↓
for this we will get
out of memory error

Ex:- Runtime exception

$\frac{a}{b} = \frac{200}{0}$ → Arithmetic Exception

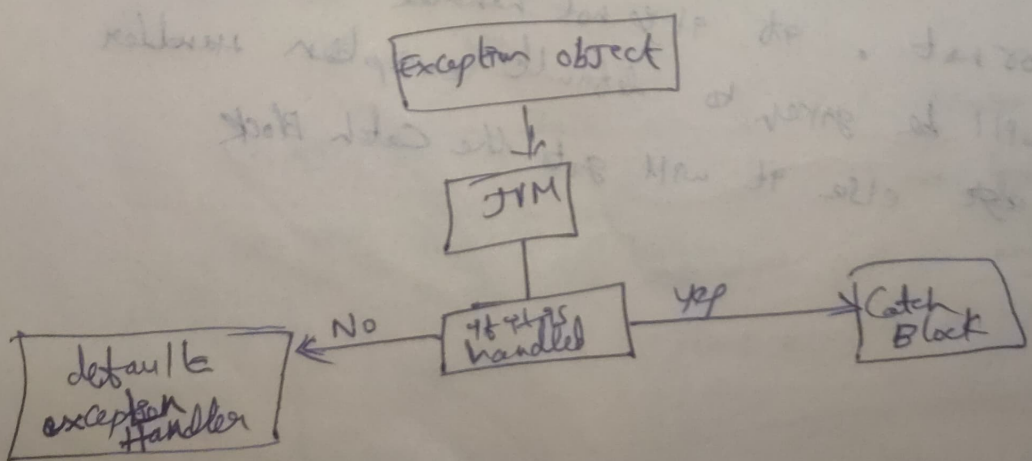
NOTE:- If exception occurs in any method that method will create the exception object and give it JVM, JVM will check whether the exception has been handled in that method or not. If it is not handled the exception obj will be given to default exception Handler else it will goto the catch Block

→ To handle exception which ever will be the risky statement in the program enclose it with try block

Exception class is parent of all the exception classes

Ex:-

```
try{
int a =
int b =
  ry = a/b; → if the exception occurs here
  s.o.p(ry); } then line won't execute
  .
  .
}
catch(Exception e){
  —
  —
}
```



class alpha division

void division()

a = ;

b = ;

ry = a/b;

sop(ry)

class beta

void dummy()

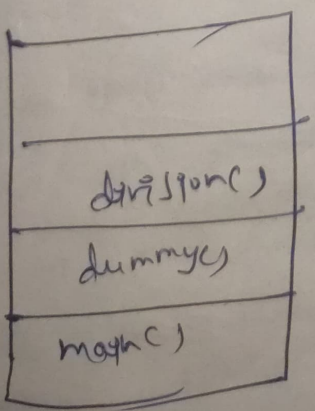
Alpha alpha = new Alpha();

alpha.division();

class game

main()

beta.dummy();



the exception occurs in the division method it is not handled then the exception object will not be sent to exception handler instead it is sent to the caller it is not handled in the caller also then it is sent to default

When an exception occurs

1. Handle the Exception (try – catch)
2. Ducking the Exception (throws)
3. Rethrowing An exception (try, catch, throw, throws, finally)

✓ throws => ~~to work with~~ checked exception
✓ throw => ~~to work with~~ unchecked Exception

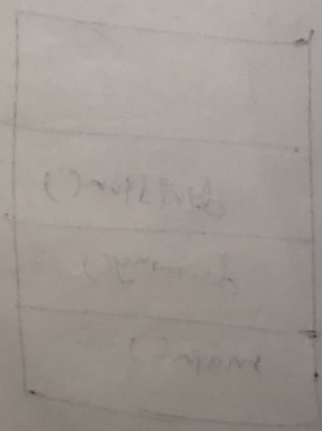
checked
↓
being checked during compile time
↓
compiler will nobby on this case ~~to~~ that an exception may occur during run time

unchecked
↓
not being checked during compile time

Ducking the exception

```
division() throws Exception
{
rq = a/b;
}

main()
{
try
division();
}
```



ducking means that we are ignoring the handling part assigning that responsibility to the caller by using throw keyword so that the caller may aware that he needs to handle it

throw

Rethrowing the Exception

→ After Handling the exception object is propagated to the caller

→ throwing the handled exception object

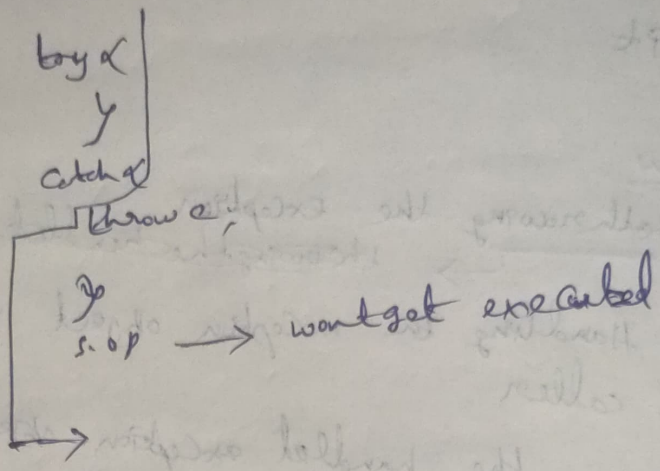
```
dosomething() throws Exc
{
  a/b
}
catch(Exc e){
  sop();
  throw e;
}

main(){
  try{
    dosomething();
  }
}
```

NOTE :- In both ducking & rethrowing we should use throws keyword

If we don't handle, the Exc object will be thrown automatically to the caller until it reaches the parent class

NOTE:-
Lines Below the throw keyword will not be executed & if those are important put them on finally block



return vs finally

```
int disp(){
    try{
        sop("disp");
    return 10;
    }
    finally
    {
        s.op("Disp final");
    }

main(){
sop(disp());
}
```

o/p:-
disp
Disp final
10.

finally block will dominate return

System.exit() vs finally

↳ terminating JVM

```
disp(){
  try{
    sop("Disp");
    System.exit(0);
  }finally
  {
    sop("Raja");
  }
}
o/p :- Disp
main(){
  disp();
}
```

System.exit() will dominate finally

throw vs finally

finally will dominate throw

only try & finally is a possible combination

Date :- 26/11/2022

Exception handling part 2 Cont...

what does the Exception object contain ?

if exception occurs in alpha method

exc obj

Name of the Exception : ArithmeticExc
Description of the Exc : / by zero
stack trace of the Exception : demo.alpha(Launch.java:22)
Launch.main(Launch.java:7)

Methods to print Exception Information

1. e.getMessage()
   → prints the description of the exception
   ex:- / by zero

2. e.toString()
   → prints the name and description of the exception
   ex:- ArithmeticException: / by zero

3. e.printStackTrace()
   → prints the name, description and the stack trace of the Exception

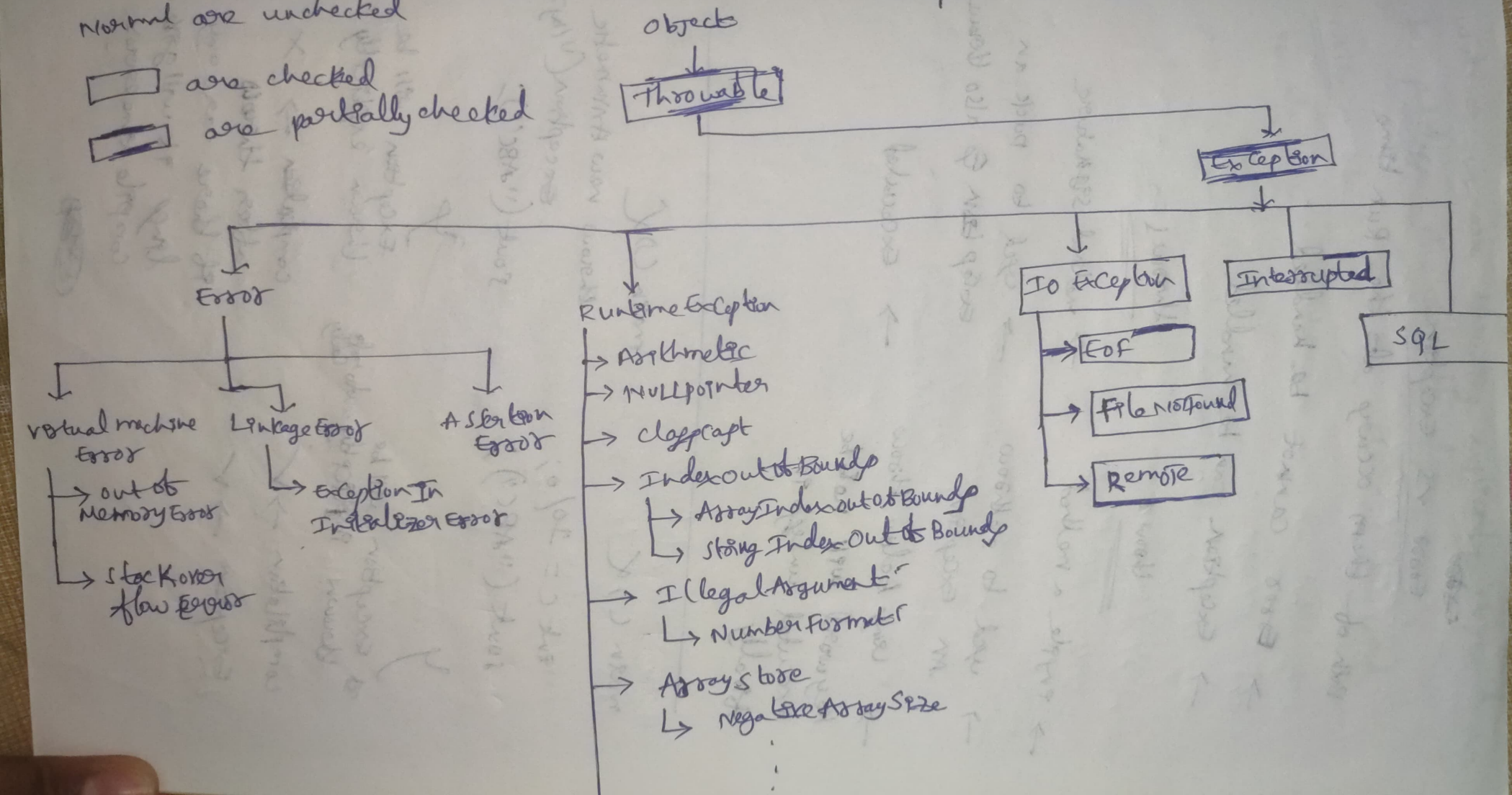
Hierarchy of Exceptions

Normal are unchecked

are checked

are partially checked

Exception → this class is partially checked



Error vs Exception

Both of them occurs at the Run time

→ Error cannot be handled

→ Exception can be handled

throw

→ purple a method

→ used to rethrow an Exception

→ code below throw keyword will not be executed unless in finally

throws

→ method signature

→ used to tuck an Exception & also throws

→ Exceeded

```
main ()
{
    int c = 10/0;
    sout ("ABC");
}
```

Exception will be thrown implicitly automatically

compilation → ✓

Exception → ✓

```
main ()
{
    throw new ArithmeticException("/ by zero");
    sout ("ABC");
}
```

Exception will be thrown explicitly

compilation → ✗ after throw if there are any lines it will give compile time error

whatever we are throwing explicitly it should be subclass of Throwable class

NOTE:- We cannot use throws keyword in the class signature

Date:- 06/08/2023

customexception / user defined exceptions

using this we can have our own exception and message,

We pass a string to the constructor of super class (Exception class) that can be obtained using getMessage() method

ex:-

```
class InvalidPasswordException extends Exception{
    public InvalidPasswordException(String exc){
        super(exc);
    }
}
```

Here InvalidPasswordException will call Exception class constructor → which will call Throwable class constructor

```
String detailMsg;
public Throwable(String exc){
    detailMsg = exc;
}
```

Throwable class will be having instance variable "detailMsg" so, our custom message will be set to that variable.

→ An example with nested try catch with custom Exceptions

```
class ATM {
public long atmNum = 123456;
public int password = 4510;

    int userNum;
    int pw;
public void input() {
        Scanner sc = ---;
        sout("Enter acctnum");
    userNum = sc.nextInt();
        sout("Enter pw");
    pw = sc.nextInt();
    }
public void verify() {
        try{if(atmNum == userNum && password == pw){
            sout("Collect your cash");
        }else{
            sout("Invalid credentials");
            throw new InvalidCredExc("Invalid pw");
        }}
    }
}
```

// custom Exc

class InvalidCredExc extends Exception {

InvalidCredExc (String msg) {

super(msg);

}

}

Bank will allow user to renter the PW
3 times, else the card will be Blocked

class Bank {

public void Initiate () {

ATM atm = new ATM ();

try {

atm. pinput ();

atm. verify ();

}

catch (InvalidCredExc e) {

try {

atm. input ();

atm. verify ();

}

catch (InvalidCredExc exc) {

sout (exc. getMessage ());

sout ("card is blocked");

System. exit (0);

}

}

}

```
main() {
    Bank b = new Bank();
    b.applicable();
}
```

Try with Resources

usually we use finally to close resources like file reader, buffered writer etc...

eg:-

```
BufferReader br = null;
FileReader fr = null;
try {
    fr = new FileReader("Ram.txt");
    br = new BufferReader(fr);
    ---;
} catch (Exception Ex) {
    ---;
} finally {
    if (br != null) {
        br.close();
    }
    if (fr != null) fr.close()
}
```

like this wherever we want to close the resources we use a piece of code everywhere

In JDK 1.7 they have come up with a resolution for this →

Try with Resource

eg:- try (BufferedReader br = new BufferedReader(new FileReader("Ravi.Txt"))){

use the resource - - -

}catch (Exception Ex){

- - -

}

the resources which are part of try block gets closed automatically after the completion of try block.

It increases readability and reduces redundant code

we can declare 'n' number of resources

try (R1; R2; R3 - - -)

NOTE:- We should only use the try with Resource for the classes which implements 'Autocloseable' class.

ex:-

```
interface AutoCloseable {
    public abstract void close();
}

public class BufferReader implements AutoCloseable {
    public void close() {
        ---
    }
}
```

→ `try (String s = new String("Rahul")) {`

```
}
```

o/p → compile time error
because String doesn't implements AutoCloseable

→ `try (BufferReader br = ---) {`

```
    br = ---;
}
```

o/p → compile time error
by default all the try resources are declared as final

NOTE :- All java.io classes and java.sql classes implements AutoCloseable interface